

---

# Discovering Cooperative Pipelines: Autoresearch for Sequential Social Dilemmas

---

Anonymous Author(s)

Affiliation

Address

email

## Abstract

1 We study *two-level autoresearch* for cooperation: an outer-loop AI agent au-  
2 tonomously redesigns the inner-loop pipeline of an LLM policy-synthesis system  
3 for multi-agent Sequential Social Dilemmas (SSDs). A *researcher agent*  $\mathcal{R}$  (run as  
4 a coding agent) reads the inner-loop source code, edits system prompts, feedback  
5 functions, helper libraries, and iteration logic, runs evaluations, and decides what  
6 to keep, following the *autoresearch* paradigm. Across two games (Cleanup and  
7 Gathering), two policy-synthesizer LLMs, and two welfare objectives (utilitarian  
8 efficiency and Rawlsian maximin), the researcher reliably exceeds hand-designed  
9 baselines, sharply tightens run-to-run variance, and outperforms prompt-only opti-  
10 mization. The discovered pipelines are objective-dependent: only under maximin  
11 does the researcher independently author synthesizer pipelines that inject an ex-  
12 plicit *fairness mechanism*, a class of mechanism absent from its own (objective-  
13 agnostic) system prompt and from all efficiency-optimized pipelines. This sup-  
14 ports an *information-design* reading in which the researcher chooses what to reveal  
15 to the boundedly rational synthesizer as a function of the welfare objective. Code  
16 at <https://github.com/anon590/autoresearch-social-dilemmas>.

## 17 1 Introduction

18 Sequential Social Dilemmas (SSDs) [1] are the multi-agent analogue of the prisoner’s dilemma in  
19 temporally rich Markov games: individually rational play leads to collectively suboptimal outcomes  
20 through pollution, over-harvesting, or open conflict. Standard multi-agent reinforcement learning  
21 (MRL) struggles in this regime due to credit assignment, non-stationarity, and large joint action  
22 spaces [2]. A complementary approach, recently introduced by Gallego [3], replaces parameter-  
23 space optimization with *algorithm-space synthesis*: a frozen LLM writes a Python policy function,  
24 evaluates it in self-play, and iteratively refines it from performance feedback. A single generation  
25 step can produce coordination logic (territory partitioning, role assignment, conditional cooperation)  
26 at a sample efficiency several orders of magnitude beyond what gradient-based MRL achieves on  
27 the same environments.

28 This shifts where the design problem lives, rather than removing it. The inner-loop pipeline that  
29 drives the synthesizer has many free parameters: which system prompt, which feedback variables,  
30 which helper functions, how many refinement steps. Each materially affects the resulting policies,  
31 and prior work tuned them by hand. A natural question follows: *can an AI agent design the pipeline?*

32 We answer affirmatively with a two-level autoresearch framework. An *outer-loop researcher agent*  
33 (Claude Opus 4.6, run as a coding agent) edits the source files of an *inner-loop policy synthesizer*  
34 (another LLM), runs evaluations on held-out seeds, and keeps modifications that improve a fixed  
35 welfare objective  $\Phi$ . The outer agent operates on an ordinary git repository (reading code, writing  
36 diffs, running shell commands, etc) without task-specific scaffolding, mirroring the autoresearch

paradigm of Karpathy [4] for single-GPU LLM pretraining. Although the inner-loop SSDs are gridworld benchmarks rather than physical systems, the outer-loop discovery process itself runs under conditions a deployed discovery agent faces: noisy multi-seed evaluations, stochastic code generation, an LLM-evaluation budget that bounds how often  $\Phi$  can be queried, and a heterogeneous code repository the agent must navigate end-to-end on its own.

Our contributions are: i) a general two-level framework that delegates the design of an LLM synthesis pipeline to a coding agent operating on a real software repository (Section 3); ii) the first instantiation of the autoresearch paradigm in a multi-agent decision-making domain, with experiments across two SSDs (Cleanup and Gathering), two policy LLMs, and two welfare objectives (utilitarian efficiency  $U$  and Rawlsian maximin  $\min_i R_i$ ) (Section 4); and iii) a mechanism-design interpretation supported by the qualitatively different pipelines the agent produces under different welfare objectives, including the autonomous insertion of explicit fairness mechanisms (usually *time-based duty rotation*) into the researcher-authored synthesizer prompts and helpers, in every maximin run and no efficiency run.

## 2 Background

We build on the iterative LLM policy synthesis framework of Gallego [3], which serves as the (frozen) inner loop of our two-level system. This section recalls the SSD formalism, the social outcome metrics, and the base synthesis loop.

### 2.1 Sequential Social Dilemmas

A Sequential Social Dilemma is a partially observable Markov game  $\mathcal{G} = \langle N, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, T, \{R_i\}_{i=1}^N, H \rangle$  with  $N$  agents, state space  $\mathcal{S}$  (the gridworld configuration), per-agent action spaces  $\mathcal{A}_i$ , transition function  $T$ , reward functions  $R_i$ , and episode horizon  $H$  [1]. Beyond the dilemma’s matrix-game structure, SSDs add temporal richness: agents must learn *when* and *where* to cooperate, not just *whether* to.

We study two canonical SSDs that capture complementary dilemma types.

**Cleanup** [5] is a *public goods provision* game ( $N=10$ ). The map has two regions: a river that accumulates waste, and an orchard where apples grow. Apples regrow only when river pollution is below threshold. Each agent can fire a cleaning beam (cost  $-1$ ) that removes waste, collect apples ( $+1$  each), or fire a tagging beam (cost  $-1$ , inflicting  $-50$  on the target and removing it for 25 steps). The dilemma: cleaning is costly but its benefits are public, so purely self-interested agents free-ride.

**Gathering** [1, 6] is a *common pool resource* game ( $N=4$ ). Agents collect shared apples on a fixed respawn timer and may fire tagging beams to temporarily remove rivals. The dilemma: agents can coexist and share resources, or aggress to monopolize them; aggression wastes time and reduces total welfare.

Both games use 8–9 discrete actions (movement, rotation, beam, stand, optionally clean) and episodes of  $H=1000$  steps. The two dilemmas differ in cost structure: asymmetric provision (cleaners pay, all benefit) vs. symmetric restraint (every agent faces the same temptation), a distinction that drives our experimental findings.

75 **Social metrics.** Following Perolat et al. [6], let  $R_i = \sum_{t=0}^{H-1} r_i^t$  denote agent  $i$ 's episode return.  
 76 We evaluate four social outcomes:

$$U = \frac{1}{H} \sum_{i=1}^N R_i \quad (\text{efficiency}) \quad (1)$$

$$E = 1 - \frac{\sum_{i,j} |R_i - R_j|}{2N \sum_i R_i} \quad (\text{equality}) \quad (2)$$

$$S = \frac{1}{N} \sum_{i=1}^N \bar{t}_i \quad (\text{sustainability}) \quad (3)$$

$$P = \frac{1}{H} \sum_{t=0}^{H-1} |\{i : \text{active}_i^t\}| \quad (\text{peace}) \quad (4)$$

77 where  $\bar{t}_i$  is the mean timestep at which agent  $i$  collects positive reward (higher  $\Rightarrow$  resources preserved  
 78 later) and  $\text{active}_i^t$  indicates that agent  $i$  is not tagged out at step  $t$ . We additionally consider the  
 79 *maximin* (Rawlsian) welfare criterion  $\min_i R_i$ , which optimizes the worst-off agent's return and  
 80 serves as the second objective in our experiments.

## 81 2.2 Iterative LLM Policy Synthesis

82 Let  $\Pi$  denote the space of *code-based policies*: deterministic functions  $\pi : \mathcal{S} \times [N] \rightarrow \mathcal{A}$  expressed  
 83 as executable Python code. Each policy has access to the full environment state and a library of  
 84 helpers (BFS pathfinding, beam targeting, coordinate transforms). This state access is a deliberate  
 85 design choice: programmatic policies operate in algorithm space rather than the reactive observation-  
 86 to-action space of neural policies, which lets a single LLM generation step encode rich coordination  
 87 logic.

88 A frozen LLM  $\mathcal{M}$  acts as the *policy synthesizer*. Given a system prompt  $p$  describing the environ-  
 89 ment API and a feedback prompt  $q_k$ , it produces a new policy

$$\pi_{k+1} = \mathcal{M}(p, q(\pi_k, \mathcal{F}_k)),$$

90 where  $\pi_k$  is the previous policy (its source code) and  $\mathcal{F}_k$  is the evaluation feedback. All  $N$  agents  
 91 execute the same program  $\pi_k$  in self-play. We stress that this is *symmetric*, not *behaviorally ho-*  
 92 *mogeneous*: since  $\pi_k$  takes `agent_id` as an argument, a single shared program can induce distinct  
 93 per-agent behaviors (cleaner vs. gatherer assignment, time-rotated duty cycles, partitioned territo-  
 94 ries; see the synthesized policies in Appendix C). What is shared is the source code; the sampled  
 95 action distributions can differ across agents. Evaluation over a set of random seeds  $S$  yields the  
 96 mean per-agent return  $\bar{r}_k$  and the social metrics vector  $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$ . Each generated  
 97 policy passes an AST-based safety check (blocking `eval`, file I/O, network access) followed by a  
 98 short smoke test; failures trigger regeneration (up to  $R$  attempts) with the error message appended  
 99 to the prompt.

100 **Feedback.** We package the previous policy's source code together with all available evaluation  
 101 signals:

$$\mathcal{F}_k = (\text{code}(\pi_k), \bar{r}_k, \mathbf{m}_k, \mathbf{d}), \quad (5)$$

102 where  $\mathbf{d}$  contains natural-language definitions of each social metric. The LLM consumes  $\mathcal{F}_k$  to  
 103 revise and improve the policy. This is a starting point; the choice of feedback content is a single  
 104 design decision within a much larger pipeline configuration space: our two-level framework (Sec-  
 105 tion 3) opens the full space to automated search.

## 106 3 Two-Level Framework

107 We introduce a two-level system where a *researcher agent*  $\mathcal{R}$  autonomously discovers configurations  
 108 that optimize the output of an inner-loop system. While we instantiate this for multi-agent policy  
 109 synthesis, the architecture is general: any pipeline where an LLM generates artifacts, evaluates  
 110 them, and iterates can serve as the inner loop. The fundamental insight is that the entire inner-loop  
 111 codebase is a designable artifact that a code-based agent can search over. Figure 1 illustrates the  
 112 architecture.

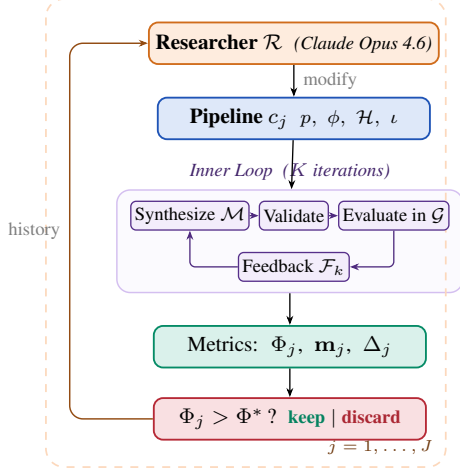


Figure 1: Two-level automated research framework (Algorithm 1).

### Algorithm 1 Two-Level Automated Research

**Require:** Game  $\mathcal{G}$ , policy synthesizer  $\mathcal{M}$ , researcher  $\mathcal{R}$ , system prompt  $p_{\mathcal{R}}$ , initial config  $c_0$ , outer iterations  $J$ , ground-truth objective  $\Phi$ , held-out seeds  $S_{\text{ho}}$

**Ensure:** Best configuration  $c^*$ , best policy  $\pi^*$

```

1:  $\pi_0^* \leftarrow \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c_0)$ 
2:  $\Phi_0 \leftarrow \text{EVAL}(\pi_0^*; \mathcal{G}, S_{\text{ho}})$ 
3:  $\text{history} \leftarrow \{(c_0, \Phi_0, \mathbf{m}_0, \emptyset)\}$ 
4: for  $j = 1, \dots, J$  do
5:    $c_j \leftarrow \mathcal{R}(p_{\mathcal{R}}, \text{CODE}(c_{j-1}), \text{history})$ 
6:   if  $\neg \text{VALIDATECONFIG}(c_j)$  then  $\text{retry} \leq R$ 
7:      $\pi_j^* \leftarrow \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c_j)$ 
8:      $\Phi_j \leftarrow \text{EVAL}(\pi_j^*; \mathcal{G}, S_{\text{ho}})$ 
9:      $\Delta_j \leftarrow \text{DIFF}(c_{j-1}, c_j)$  // code diff
10:     $\text{history} \leftarrow \text{history} \cup \{(c_j, \Phi_j, \mathbf{m}_j, \Delta_j)\}$ 
11:  end for
12:  $j^* \leftarrow \arg \max_j \Phi_j$ 
13: return  $c_{j^*}, \pi_{j^*}^*$ 

```

Table 1: Configuration components modifiable by the researcher, with concrete edits that  $\mathcal{R}$  made in our runs. The environment simulator, ground-truth evaluation, and policy LLM weights are frozen.

|               | Scope           | Concrete edits by $\mathcal{R}$ (see appendix)                                                                                               |
|---------------|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| $p$           | System prompt   | Multi-section strategic briefing with explicit duty-rotation template (agent_id + step//T) % n (App. B.3, Listing 4)                         |
| $\phi$        | Feedback fn     | Thresholded diagnostics: “FAIRNESS ALERT” fires when $\min_i R_i < 0$ ; “DO NOT REGRESS” guard fires when $U \geq 2.5$ (App. B.4, Listing 5) |
| $\mathcal{H}$ | Helper library  | BFS-Voronoi territories with respawn-timer awareness; band-based apple zoning by agent_id (App. B.2, Listings 3, 2)                          |
| $\iota$       | Iteration logic | Per-condition $K \in \{2, 3\}$ ; $ S $ walked 5→8→12 for variance control; thinking budget 10–32k (App. B.5, Table 6)                        |

### 3.1 Configuration Space

Let  $\mathcal{C}$  denote the space of *pipeline configurations*. Each configuration  $c \in \mathcal{C}$  specifies the full inner-loop setup:

$$c = (p, \phi, \mathcal{H}, \iota) \quad (6)$$

where  $p$  is the system prompt,  $\phi$  is the feedback construction function (which metrics and diagnostics to include, how to frame it, whether to inject adaptive hints and thresholds, etc),  $\mathcal{H}$  is the helper function library (auxiliary functions for pathfinding, getting aggregates of useful environment quantities, etc.), and  $\iota$  specifies the iteration logic (number of inner iterations  $K$ , sampling strategy). Table 1 provides concrete examples. The validation pipeline is part of the frozen inner-loop infrastructure rather than a configurable component, the researcher cannot modify it in our experiments to prevent reward hacking.

The hand-designed feedback of [3] corresponds to a single fixed instantiation of  $\phi$ . Our framework opens the full configuration space to automated search.

### 3.2 Inner Loop (Policy Synthesis)

Given a configuration  $c$ , the inner loop executes  $K$  iterations of LLM policy synthesis:

$$\pi_c^* = \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c) \quad (7)$$

Each iteration  $k$  proceeds in four stages, following [3]:

1. **Synthesize.** The policy LLM  $\mathcal{M}$  receives the system prompt  $p$ , the previous policy’s source code  $\pi_{k-1}$ , and feedback  $\mathcal{F}_{k-1}$  constructed by  $\phi$ . It generates a new Python policy function  $\pi_k$  that has access to full environment state and the helper library  $\mathcal{H}$ .

- 131 2. **Validate.** The generated code undergoes AST-based safety checks (blocking dangerous oper-  
 132 ations such as file I/O and network access) followed by a short smoke test. Failures trigger  
 133 re-generation (up to  $R$  retries), with the error message appended to the prompt.
- 134 3. **Evaluate.** All  $N$  agents execute the same policy  $\pi_k$  in self-play over  $|S|$  random seeds (note the  
 135 policy is conditional on `agent_id`). The evaluation yields the mean per-agent reward  $\bar{r}_k$  and the  
 136 social metrics vector  $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$ .
- 137 4. **Feedback.** The feedback function  $\phi$  constructs the prompt for the next iteration from  
 138  $(\pi_k, \bar{r}_k, \mathbf{m}_k)$ , packaging the previous policy’s source code together with the scalar reward, the  
 139 social metrics vector, their natural-language definitions, and any adaptive diagnostics that  $\phi$  in-  
 140 jects.

141 The inner loop output is evaluated on held-out seeds via a fixed ground-truth objective:

$$\Phi(c) = \text{EVAL}(\pi_c^*; \mathcal{G}, S_{\text{held-out}}) \quad (8)$$

142 where  $\Phi$  maps from social outcomes to a scalar. We consider two alternative objectives to maximize:

$$\Phi_U = U = \frac{1}{H} \sum_{i=1}^N R_i \quad (\text{utilitarian efficiency}) \quad (9)$$

$$\Phi_{\min} = \min_i R_i \quad (\text{Rawlsian maximin}) \quad (10)$$

143  $\Phi_U$  rewards collective throughput and is indifferent to how reward is distributed across agents,  
 144 whereas  $\Phi_{\min}$  instead optimizes for the worst-off agent, pressuring the researcher toward configu-  
 145 rations that distribute the cost of cooperation.

### 146 3.3 Outer Loop (Automated Research)

147 The researcher agent  $\mathcal{R}$  iteratively modifies the pipeline configuration. Following the autoresearch  
 148 paradigm [4],  $\mathcal{R}$  operates on the inner-loop codebase as a modifiable artifact, proposing changes,  
 149 observing outcomes, and refining. The procedure is formalized in Algorithm 1.

150 At each outer iteration  $j$ , the researcher  $\mathcal{R}$  receives: i) the **full source code** of the current config-  
 151 uration  $c_{j-1}$  (prompts, feedback construction, helpers, iteration logic); ii) the **experiment history**:  
 152 for each prior iteration, the code diff  $\Delta_i$ , ground-truth score  $\Phi_i$ , and social metrics vector  $\mathbf{m}_i$ ; iii)  
 153 the **environment source code** (read-only), enabling the researcher to reason about game mechanics.  
 154 The researcher proposes a new configuration  $c_j$  by generating code modifications. Concretely,  $\mathcal{R}$   
 155 is a coding agent (Claude Code CLI) that operates on a real software repository: it reads and edits  
 156 Python source files, runs shell commands, inspects evaluation outputs, and commits changes to a  
 157 dedicated git branch, following the same workflow a human researcher would follow.

### 158 3.4 Connection to Automated Mechanism Design

159 The two-level structure admits a mechanism design interpretation. The researcher  $\mathcal{R}$  acts as a *mech-*  
 160 *anism designer*: it controls the information structure (what metrics to reveal, how to frame them),  
 161 the action space (what helper functions are available), and the incentive structure (how feedback is  
 162 presented) under which the policy synthesizer  $\mathcal{M}$  operates. The synthesizer acts as the *agent* within  
 163 the designed mechanism.

164 This connects to the automated mechanism design literature [7], where a principal designs rules to  
 165 induce desired behavior from self-interested agents. In our setting:

- 166 • The **principal** is the researcher  $\mathcal{R}$ , optimizing the ground-truth objective  $\Phi$ .
- 167 • The **agent** is the synthesizer  $\mathcal{M}$ , optimizing per-agent reward as instructed.
- 168 • The **mechanism** is the configuration  $c$ : prompts, feedback, helpers, iteration logic.
- 169 • The **outcome** is the social welfare of the resulting multi-agent policy  $\pi_c^*$ .

170 A crucial distinction from classical mechanism design:  $\mathcal{M}$  follows instructions but has *bounded*  
 171 *rationality* in the sense that its ability to synthesize effective policies depends on the information  
 172 and tools provided. The researcher’s task is thus closer to *information design* [8]: choosing what  
 173 to reveal to help  $\mathcal{M}$  navigate the cooperation–defection tension. Our experiments (Section 4) show  
 174 that the researcher designs qualitatively different information structures depending on the welfare  
 175 objective  $\Phi$ , supporting this interpretation empirically.

## 4 Experiments

### 4.1 Setup

We conduct 12 autonomous researcher runs across a factorial design. The researcher agent  $\mathcal{R}$  is Claude Opus 4.6, invoked via the Claude Code CLI as a coding agent. Each run operates on a dedicated git branch of a real Python codebase:  $\mathcal{R}$  edits source files in `pipeline/`, executes evaluation scripts, reads metric outputs, and iterates, without human intervention.

**Design.** For Cleanup: 2 policy LLMs  $\times$  2 objectives  $\times$  2 replications = 8 runs. For Gathering: 2 policy LLMs  $\times$  1 objective  $\times$  2 replications = 4 runs. Maximin runs are unnecessary for Gathering because efficiency optimization alone achieves close to perfect equality.

**Models.** Policy synthesizer  $\mathcal{M}$ : Gemini 3.1 Pro (Google) or Claude Sonnet 4.6 (Anthropic), to recent state-of-the-art LLMs. Both use extended thinking. We additionally test with Gemma 4 26B-A4B-IT (Google), a smaller open-weight model, to probe the framework’s behavior when the policy synthesizer has substantially lower capability (Appendix A.1).

**Baselines.** The hand-designed feedback configuration from prior work [3] serves as the initial pipeline  $c_0$  for all runs. On Cleanup ( $N=10$ ), this baseline achieves  $U=1.93/2.70$  (mean/max) with Gemini and  $U=0.86/1.56$  with Sonnet. We additionally compare against GEPA [9], an automated prompt optimization method that iteratively refines the system prompt via LLM reflection. GEPA optimizes only the system prompt  $p$ , whereas our researcher modifies the full pipeline  $(p, \phi, \mathcal{H}, \iota)$ . We give GEPA a matched compute budget: the same number of optimization steps as outer iterations used by our method, so all in all both methods use a comparable number of environment evaluations.

### 4.2 Results

Table 2 presents the main results, comparing our autoresearch framework against the hand-designed baseline of [3] and GEPA [9].

**Finding 1: The researcher reliably improves over hand-designed baselines and outperforms prompt-only optimization.** Every run improves substantially regardless of starting point (Figure 2). On Cleanup, autoresearch lifts both LLMs to  $U \approx 3.1$ – $3.2$  from baselines of 1.93 (Gemini) and 0.86 (Sonnet), nearly closing the gap between them; on Gathering, all four runs converge to  $U \in [2.47, 2.52]$  from baselines spanning 0.03–2.42. Run-to-run spread is tight (gaps within 0.05 on Cleanup- $\Phi_U$ ), suggesting the researcher reliably finds the performance ceiling of each policy LLM via the pipeline modifications it discovers (helpers, prompts, and feedback; see appendices B.2–B.4 for a selection of them). At matched environment queries, autoresearch beats GEPA by 2–3 $\times$  on Cleanup (both  $\Phi_U$  and  $\Phi_{\min}$ ) and 20% on Gathering, with the gap widening for the weaker policy LLM: GEPA-Sonnet under  $\Phi_{\min}$  can collapse to a pathological “everyone cleans, nobody eats” regime ( $U=-2.87$ ,  $E=1.00$ ), while autoresearch-Sonnet reaches  $\min_i R_i \approx 200$  reliably. Modifying the full pipeline, not just the prompt, is what closes these gaps.

**Finding 2: No efficiency–fairness tradeoff in Cleanup (Gemini).** Maximin-optimized Gemini pipelines sacrifice only 1% efficiency ( $U$ : 3.16 vs. 3.20) while achieving near-perfect equality ( $\bar{E}$ : 0.98 vs. 0.55) and transforming maximin from deeply negative baselines (−99 to −84) to  $\min_i R_i = 290$  (Figure 3). The researcher discovers *fair duty rotation* (Listing 7; primed by the prompt rewrite of Listing 4)—time-based cycling using `agent_id` and `env._step_count`—simultaneously improving worst-off welfare *and* collective output. Because cleaning is a public good, distributing the cleaning cost fairly ensures enough cleaners to sustain apple production.

For Sonnet, there is a moderate tradeoff: efficiency drops from 3.12 to 2.57 under maximin optimization. The gap reflects Sonnet’s harder time implementing complex coordination mechanisms (role rotation, zone assignment) from strategic hints alone.

**Finding 3: Game structure determines whether fairness requires explicit optimization.** In Cleanup, where cleaning costs are borne asymmetrically (cleaners pay −1, free-riders collect apples), baseline equality ranges from  $E=0.04$  to 0.62, and maximin optimization is required to reach

Table 2: Results. Mean / max across the runs per condition. **Bold** marks the best mean per metric within each Policy LLM  $\times$  Game sub-block. Baseline: unmodified, hand-designed pipeline from [3]. GEPA: automated prompt optimization [9] with matched compute budget.

| Policy LLM                                | Target        | $U$               | $E$               | $\min_i R_i$    |
|-------------------------------------------|---------------|-------------------|-------------------|-----------------|
| <i>Cleanup — Baseline</i>                 |               |                   |                   |                 |
| Gemini 3.1 Pro                            | —             | 1.93/2.70         | 0.17/0.62         | −159/−84        |
| Sonnet 4.6                                | —             | 0.86/1.56         | −0.02/0.25        | −151/−59        |
| <i>Cleanup — GEPA [9]</i>                 |               |                   |                   |                 |
| Gemini 3.1 Pro                            | $\Phi_U$      | 1.34/1.37         | −0.10/−0.05       | −149/−126       |
|                                           | $\Phi_{\min}$ | 1.76/2.76         | 0.90/0.96         | 143/245         |
| Sonnet 4.6                                | $\Phi_U$      | 1.04/1.20         | −0.59/−0.21       | −164/−126       |
|                                           | $\Phi_{\min}$ | −0.63/1.60        | 0.77/1.00         | −147/6          |
| <i>Cleanup — Two-level Autoresearch</i>   |               |                   |                   |                 |
| Gemini 3.1 Pro                            | $\Phi_U$      | <b>3.20</b> /3.25 | 0.55/0.61         | −211/−182       |
|                                           | $\Phi_{\min}$ | 3.16/3.19         | <b>0.98</b> /0.98 | <b>290</b> /296 |
| Sonnet 4.6                                | $\Phi_U$      | <b>3.12</b> /3.14 | 0.66/0.70         | −196/−146       |
|                                           | $\Phi_{\min}$ | 2.57/2.93         | <b>0.91</b> /0.97 | <b>179</b> /200 |
| <i>Gathering — Baseline</i>               |               |                   |                   |                 |
| Gemini 3.1 Pro                            | —             | 2.04/2.42         | 0.90/0.98         | 412/571         |
| Sonnet 4.6                                | —             | 0.03/0.03         | 0.54/0.54         | 0/0             |
| <i>Gathering — GEPA [9]</i>               |               |                   |                   |                 |
| Gemini 3.1 Pro                            | $\Phi_U$      | 2.08/2.35         | 0.94/0.96         | 436/518         |
| Sonnet 4.6                                | $\Phi_U$      | 1.20/1.23         | 0.63/0.71         | 86/123          |
| <i>Gathering — Two-level Autoresearch</i> |               |                   |                   |                 |
| Gemini 3.1 Pro                            | $\Phi_U$      | <b>2.49</b> /2.51 | <b>0.98</b> /0.98 | <b>582</b> /582 |
| Sonnet 4.6                                | $\Phi_U$      | <b>2.52</b> /2.52 | <b>0.96</b> /0.98 | <b>576</b> /597 |

224  $E > 0.9$ . In Gathering, where all agents face a symmetric landscape, efficiency optimization alone  
225 achieves  $E > 0.94$  across all 4 runs: no separate maximin runs are needed. This generalizes: in  
226 *provision* dilemmas with asymmetric costs, fairness requires designed mechanisms (role rotation,  
227 duty sharing); in *restraint* dilemmas with symmetric costs, fairness emerges as a free byproduct of  
228 efficient coordination. The researcher independently discovers this, it creates role differentiation  
229 pipelines only for Cleanup, and pure spatial-coordination pipelines for Gathering.

230 **Finding 4: Convergent discovery of qualitatively different strategies per objective.** Despite  
231 fully independent runs, the researcher converges on the same core strategies within each condition  
232 (Table 3, Appendix A). In Cleanup, waste-counting helpers and spatial zone partitioning appear  
233 across all runs. The qualitative dividing line is the presence of an explicit *fairness mechanism*,  
234 which appears in 4/4 maximin runs but 0/4 efficiency runs. In 3/4 maximin runs (both Gemini runs  
235 and one Sonnet run, Listings 7, 8) the researcher writes *time-based role rotation* into the synthe-  
236 sizer prompt; in the remaining maximin run the researcher writes a structurally distinct “collective  
237 threshold” mechanism in which all agents synchronously switch between cleaning and collecting  
238 based on `waste_fraction(env)` (achieving comparable maximin without an agent-index phase).  
239 Under efficiency optimization, the researcher instead writes *static* role assignment (some agents al-  
240 ways clean), producing high collective output at the cost of equality (Listing 6). In Gathering, the  
241 researcher discovers BFS-Voronoi territory partitioning and respawn-timer awareness (Listings 3, 9),  
242 with no role differentiation: optimization is purely spatial (who collects which apples) and temporal  
243 (respawn-aware positioning).

244 The convergence here is at the level of *which artifacts*  $\mathcal{R}$  injects into  $p, \phi, \mathcal{H}$ . Once the researcher-  
245 authored prompt contains a rotation template (e.g., Listing 4), the downstream synthesizer’s imple-  
246 mentation is unsurprising. The non-trivial claim is that  $\mathcal{R}$  writes such a template only under  $\Phi_{\min}$ ,  
247 never under  $\Phi_U$ , despite the researcher system prompt  $p_{\mathcal{R}}$  being identical across objectives and con-  
248 taining no rotation formula, no objective-conditional guidance, and no link between any strategy  
249 class and either welfare criterion (Listing 1, Appendix B.1).

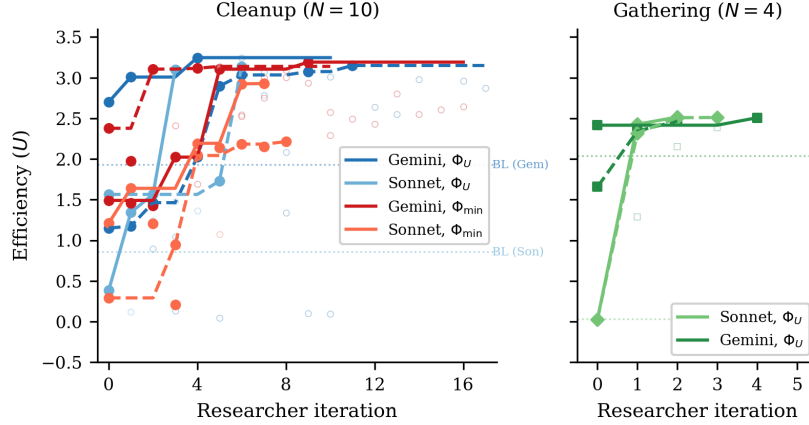


Figure 2: Efficiency ( $U$ ) across researcher iterations for all 12 runs. *Left*: Cleanup with 8 runs across 2 LLMs  $\times$  2 objectives (solid lines = kept experiments, open circles = discarded). Dashed horizontal line: hand-designed baseline from [3]. All runs converge to  $U \approx 3.1$ – $3.2$  despite diverse starting points. *Right*: Gathering with 4 runs (2 per LLM). All converge to  $U \approx 2.5$  within 2–4 iterations despite baselines ranging from 0.03 to 2.42.

250 **Common failure modes.** Several patterns led to discarded autoresearch iterations across all runs:  
 251 (1) *over-prescription*: adding too many strategic hints confuses the policy LLM, causing generation  
 252 failures; (2) *iteration regression*:  $K=4$  or  $K=5$  inner iterations often cause catastrophic regression  
 253 as the LLM “over-refines” a working policy; (3) *feedback overload*: verbose per-agent diagnostics  
 254 cause over-correction rather than gradual improvement.

## 255 5 Related Work

256 **Sequential social dilemmas.** SSDs were introduced by Leibo et al. [1] (Gathering) and extended  
 257 to public-goods settings by Hughes et al. [5] (Cleanup); Perolat et al. [6] formalized the social  
 258 outcome metrics ( $U, E, S, P$ ) used here. Subsequent work studies inequity aversion, intrinsic moti-  
 259 vation, and reputational mechanisms for promoting cooperation in MARL agents. We complement  
 260 this line by automating the search for cooperative *programs* rather than evolving neural policies, and  
 261 by treating the welfare objective  $\Phi$  as a designable parameter that the outer agent optimizes for, ob-  
 262 taining qualitatively different cooperative behaviors (static role assignment vs. duty rotation) under  
 263 different objectives without changing the environment.

264 **LLMs for policy and program synthesis.** FunSearch [10] evolves programs for combinato-  
 265 rial discovery; Eureka [11] synthesizes reward functions from environment source code; Voy-  
 266 ager [12] and Code as Policies [13] generate executable skill code for single-agent embodied control;  
 267 ReEvo [14] evolves heuristics with reflective feedback. These works target single-agent settings or  
 268 non-strategic optimization. Gallego [3], on which our inner loop is based, extended LLM program  
 269 synthesis to the multi-agent SSD setting, where one program must coordinate  $N$  self-play copies.  
 270 Our contribution is one level above: rather than tuning the synthesizer for one task, we let an outer  
 271 agent rewrite the synthesis pipeline itself.

272 **LLM reflection and prompt optimization.** Reflexion [15], Self-Refine [16], OPRO [17], and  
 273 GEPA [9] demonstrate that structured verbal feedback loops improve LLM outputs; ERL [18] in-  
 274 ternalizes such reflection via self-distillation. These methods optimize the prompt or the model’s  
 275 reasoning trajectory in isolation. Our framework instead modifies the entire surrounding pipeline  
 276 (system prompt, feedback construction, helper library, iteration logic) making prompt optimization  
 277 one component of a strictly larger search space. The empirical gap is large: Section 4 shows full-  
 278 pipeline autoresearch achieving  $3\times$  higher efficiency in Cleanup and 37% higher in Gathering than  
 279 GEPA at matched compute, with run-to-run spread an order of magnitude tighter on the harder  
 280 fairness objective.



**Automated AI research.** A small but growing body of work delegates the design of ML pipelines to LLM coding agents. Karpathy’s autoresearch [4] runs a coding agent that modifies `train.py` for nanoGPT pretraining and is rewarded for validation loss. Our system shares this autoresearch architecture (coding agent + frozen evaluation harness + diff-based history) but targets a different inner loop (multi-agent policy synthesis instead of single-model pretraining) and a different objective (multi-agent social welfare instead of validation bits-per-byte). To our knowledge this is the first instantiation of the autoresearch paradigm in a multi-agent decision-making domain.

**Automated mechanism and information design.** Classical automated mechanism design [7] computes optimal allocation rules for self-interested agents under explicit incentive constraints, while information design [8] studies how a principal commits to a signaling policy that shapes a receiver’s behavior. Our outer agent solves a related but distinct problem:  $\mathcal{M}$  is not strategically deceptive (it follows instructions) but is *boundedly rational*, so the researcher must decide what information, helpers, and structure to expose so that  $\mathcal{M}$  writes policies achieving the principal’s welfare goal. Section 4 shows that the pipelines  $\mathcal{R}$  produces are genuinely a function of the welfare objective, supporting this information-design framing empirically.

## 6 Discussion and Conclusion

We have presented a two-level framework where a coding agent autonomously discovers pipeline configurations that improve the output of an inner-loop LLM system. Applied to multi-agent policy synthesis in social dilemmas, the researcher agent reliably exceeds hand-designed baselines across multiple independent runs, and converges on qualitatively similar strategies within each game-objective condition. The framework requires no domain-specific scaffolding: the agent operates on a standard software repository using the same tools (file editing, shell commands, git) available to a human researcher.

**Mechanism design in action.** Our results empirically support the mechanism design interpretation of Section 3.4. The researcher acts as an information designer: under  $\Phi_U$ , it reveals efficiency-oriented information (waste counts, zone assignments) that guides  $\mathcal{M}$  toward productive but unequal role allocation (Listing 6). Under  $\Phi_{\min}$ , it additionally reveals fairness-oriented structure such as rotation schedules and per-agent equity feedback (Listings 4, 5), guiding  $\mathcal{M}$  toward egalitarian coordination.

**Two types of dilemma.** The Cleanup–Gathering contrast reveals a structural insight about social dilemmas: *provision* dilemmas (asymmetric costs, one agent’s contribution benefits all) require explicit fairness mechanisms, while *restraint* dilemmas (symmetric costs, each agent faces the same temptation) achieve fairness naturally when coordination improves. This distinction, well-known in the common-pool resource literature, is rediscovered here by an autonomous AI agent, which suggests it is a robust structural feature of these environments.

**Human oversight and audit.** Our system is fully autonomous by design, but its architecture offers natural affordances for human-in-the-loop oversight. Because the researcher  $\mathcal{R}$  operates on a standard git repository, a domain expert can audit the full history of code diffs  $\Delta_j$  alongside their metric outcomes  $(\Phi_j, \mathbf{m}_j)$ . The keep/discard decision (Algorithm 1, line 11) is a lightweight intervention point: a human could override the accept threshold, veto modifications that introduce undesirable mechanisms (e.g., exploitative role assignments), or steer the researcher toward under-explored regions of the configuration space by editing the experiment history. More broadly, the separation between the researcher  $\mathcal{R}$  and the frozen evaluation  $\Phi$  creates a *delegation boundary*: the human defines *what* to optimize (the welfare objective), while the agent decides *how*. This mirrors the expert-in-the-loop design patterns identified in deployment-oriented work on AI agents for discovery, where trust calibration depends on the legibility of the agent’s decision trail rather than on real-time supervision.

**Future work.** Several directions emerge. First, we are interested in applying the two-level framework to other LLM-driven pipelines (code optimization, scientific experiment design, infrastructure tuning), where the same architecture applies with a different inner loop and objective  $\Phi$ ; next, adversarial objectives can be intriguing, to test whether the researcher discovers exploitative pipeline

configurations, exposing reward hacking risks; and asymmetric programs: extend to settings where agents run *different* source code (the present setup is symmetric in code but already supports heterogeneous behavior via `agent_id`; relaxing the shared-program constraint enables richer per-agent specialization and partial-information settings).

## References

- [1] Joel Z Leibo, Vinicius Zambaldi, Marc Lanctot, Janusz Marecki, and Thore Graepel. Multi-agent reinforcement learning in sequential social dilemmas. In *Proc. 16th Conference on Autonomous Agents and MultiAgent Systems*, pages 464–473, 2017.
- [2] Lucian Buşoniu, Robert Babuška, and Bart De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008.
- [3] Víctor Gallego. Cooperation and exploitation in llm policy synthesis for sequential social dilemmas. *arXiv preprint arXiv:2603.19453*, 2026.
- [4] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. <https://github.com/karpathy/autoresearch>, 2026.
- [5] Edward Hughes, Joel Z Leibo, Matthew Phillips, Karl Tuyls, Edgar A Dueñez-Guzmán, Antonio García Castañeda, Iain Dunning, Tina Zhu, Kevin R McKee, R. Koster, Heather Roff, and Thore Graepel. Inequity aversion improves cooperation in intertemporal social dilemmas. In *Neural Information Processing Systems*, 2018.
- [6] Julien Perolat, Joel Z Leibo, Vinicius Zambaldi, Charles Beattie, Karl Tuyls, and Thore Graepel. A multi-agent reinforcement learning model of common-pool resource appropriation. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [7] Vincent Conitzer and Tuomas Sandholm. Complexity of mechanism design. In *Proc. 18th Conference on Uncertainty in Artificial Intelligence*, pages 103–110, 2002.
- [8] Emir Kamenica and Matthew Gentzkow. Bayesian persuasion. *American Economic Review*, 101(6):2590–2615, 2011.
- [9] Lakshya A Agrawal et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2026.
- [10] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- [11] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- [12] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.
- [13] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500, 2023.
- [14] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

- [15] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [17] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024.
- [18] Taiwei Shi, Sihao Chen, Bowen Jiang, Linxin Song, Longqi Yang, and Jieyu Zhao. Experiential reinforcement learning. *arXiv preprint arXiv:2602.13949*, 2026.

## A Additional Results

Table 3: Strategies discovered by the researcher in Cleanup across 4 efficiency-optimized and 4 maximin-optimized independent runs. The grouped row records any explicit *fairness mechanism*; its two sub-rows decompose this into the dominant variant (time-based role rotation) and the alternative (synchronized whole-population clean/collect switching) found in one Sonnet maximin run.

| Strategy                                     | $\Phi_U$ (4 runs) | $\Phi_{\min}$ (4 runs) |
|----------------------------------------------|-------------------|------------------------|
| Waste-counting helpers                       | 4/4               | 4/4                    |
| Zone/lane partitioning                       | 3/4               | 4/4                    |
| Anti-regression feedback                     | 3/4               | 2/4                    |
| Worked policy examples                       | 2/4               | 3/4                    |
| Cleaning cost economics                      | 2/4               | 3/4                    |
| <b>Explicit fairness mechanism</b>           | <b>0/4</b>        | <b>4/4</b>             |
| <i>of which</i> : time-based role rotation   | 0/4               | 3/4                    |
| <i>of which</i> : synchronized clean/collect | 0/4               | 1/4                    |

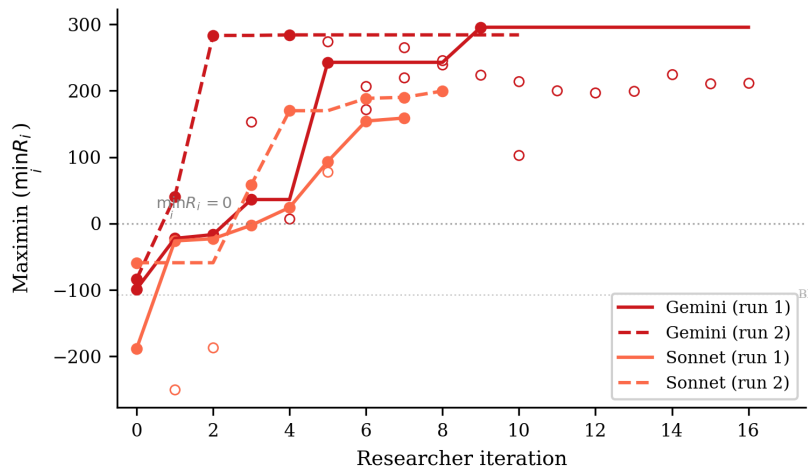


Figure 3: Maximin ( $\min_i R_i$ ) across researcher iterations for the 4 Cleanup maximin-optimized runs. All runs transform deeply negative baselines (worst-off agents losing reward) into substantially positive values. Gemini reaches  $\sim 290$  while Sonnet reaches  $\sim 160$ – $200$ . The dashed line marks  $\min_i R_i = 0$  (no agent loses reward overall).

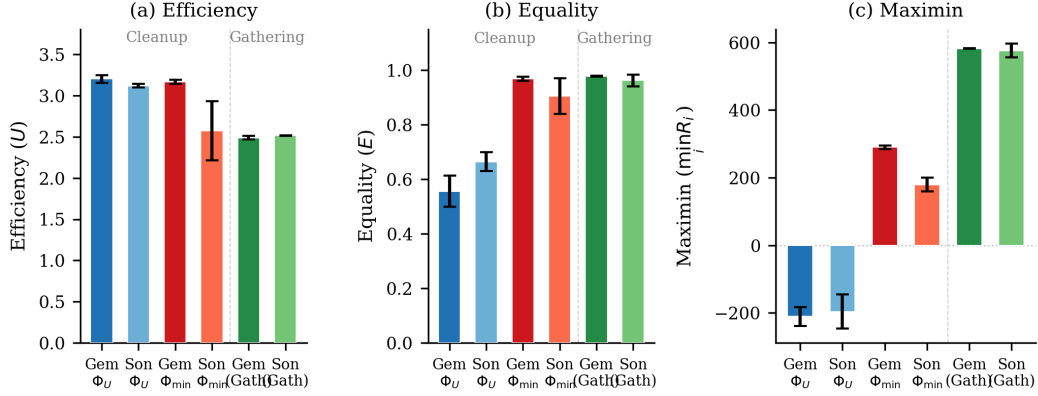


Figure 4: Final metrics across all conditions. (a) Efficiency: all conditions converge to  $\bar{U} \approx 2.5$ – $3.2$ . (b) Equality: maximin-optimized Cleanup runs achieve  $E \approx 1.0$ , while efficiency-optimized runs show  $E \approx 0.5$ – $0.7$ ; Gathering achieves high equality regardless. (c) Maximin: the sharpest contrast—efficiency optimization leaves the worst-off agent deeply negative ( $\min_i R_i \approx -200$ ), while maximin optimization transforms it to  $+290$  (Gemini) or  $+179$  (Sonnet). Gathering achieves high maximin ( $\sim 580$ ) under efficiency optimization alone. Error bars show s.d. across runs.

391 **Inner-loop averages confirm a broad-based improvement.** Figures 2 and 3 report the metric of  
 392 the *kept* inner-loop output ( $\pi_c^*$ ) at each outer iteration. A natural concern is that this could overstate  
 393 pipeline quality if the researcher is benefitting from variance: with  $K$  inner iterations per outer  
 394 step, a single lucky generation could carry the curve while the rest of the inner trajectory is noise.  
 395 Figures 5 and 6 replot the same runs with the metric averaged across *all* inner iterations of each outer  
 396 step. The trajectories ramp at essentially the same rate, with only mild attenuation: Cleanup mean  
 397 efficiency still climbs to  $\bar{U} \approx 2.5$ – $3.0$  (vs.  $3.1$ – $3.2$  for the kept policy), Gathering still saturates at  
 398  $\bar{U} \approx 2.5$ , and Cleanup mean maximin still reaches  $\min_i R_i \approx 200$ – $275$  (vs.  $179$ – $290$ ). The whole  
 399 inner-loop output distribution improves, not just its tail, consistent with the interpretation that the  
 400 researcher is shaping the synthesizer’s behavior rather than amplifying lucky draws.

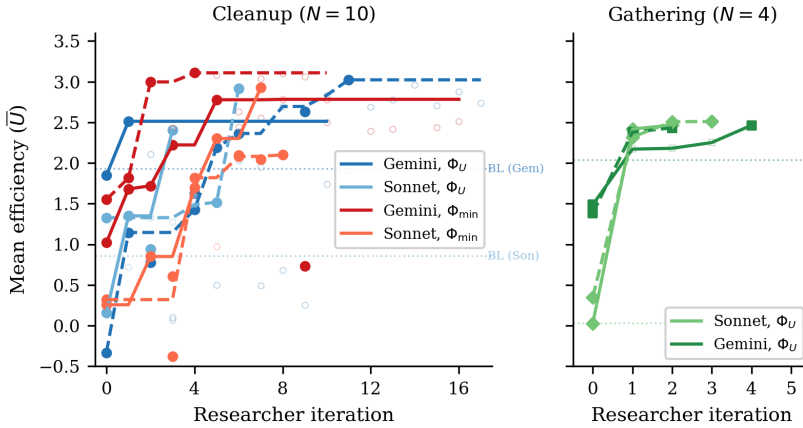


Figure 5: Mean efficiency  $\bar{U}$  averaged across all inner iterations of each outer step (compare Figure 2, which shows the kept policy only). *Left*: Cleanup ( $N=10$ ); *right*: Gathering ( $N=4$ ). The trajectories track Figure 2 closely, indicating that the researcher’s gains come from improving the entire inner-loop output distribution, not just the best of  $K$  samples.

#### 401 A.1 Smaller Open-Weight Model: Gemma 4 26B

402 We test the framework with Gemma 4 26B-A4B-IT (Google), a 26B-parameter open-weight model  
 403 substantially smaller than the frontier models in the main experiments. We run three experiment runs:

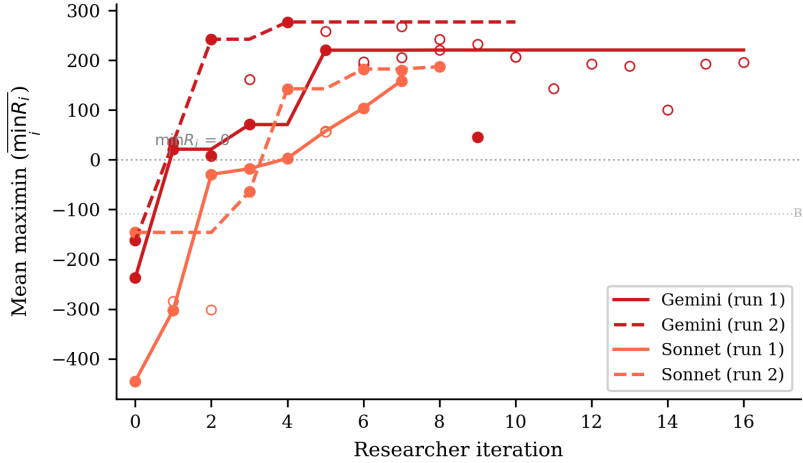


Figure 6: Mean maximin  $\overline{\min_i R_i}$  averaged across all inner iterations of each outer step for the 4 Cleanup  $\Phi_{\min}$  runs (compare Figure 3). The deeply negative baselines lift to  $\sim 200$ – $275$  mean maximin, only mildly below the kept-policy values of Figure 3. The horizontal dashed line marks  $\min_i R_i = 0$ .

two on Cleanup optimizing efficiency and maximin respectively, and one on Gathering optimizing efficiency. As with the other synthesizer models, all with Opus 4.6 as the researcher  $\mathcal{R}$ .

**Setup.** In all three experiments, Gemma starts from complete failure: the baseline pipeline produces  $U = -10.0$  on Cleanup (agents CLEAN-spam without collecting apples) and  $U = 0.0$  on Gathering (broken BFS calls), compared to  $U = 1.93/0.86$  (Gemini/Sonnet on Cleanup) and  $U = 2.04/0.03$  (Gemini/Sonnet on Gathering). The researcher ran  $J = 10$ – $18$  outer iterations per condition.

**Results.** Table 4 presents the best configurations discovered. Under efficiency optimization, the researcher achieves  $U = 0.87$ , a dramatic recovery from the broken baseline but far below frontier models ( $U \approx 3.2$ ). The researcher compensates for the model’s limited capability by reducing inner-loop iterations to  $K = 1$  (avoiding the regression that plagued  $K \geq 2$  runs) and providing highly structured worked examples with explicit cleaning-role assignment.

Table 4: Autoresearch with Gemma 4 26B-A4B-IT. Single run per condition. Baseline: pipeline from [3].

| Game      | Target        | $U$         | $E$               | $\min_i R_i$ | Keep rate |
|-----------|---------------|-------------|-------------------|--------------|-----------|
| Cleanup   | Baseline      | -10.0       | 1.00 <sup>†</sup> | -1000        | —         |
|           | $\Phi_U$      | 0.87        | -1.11             | -371         | 6/19      |
|           | $\Phi_{\min}$ | <b>1.71</b> | <b>0.94</b>       | <b>137</b>   | 9/17      |
| Gathering | Baseline      | 0.0         | 1.00 <sup>†</sup> | 0            | —         |
|           | $\Phi_U$      | <b>2.44</b> | <b>0.98</b>       | <b>580</b>   | 4/11      |

<sup>†</sup>Equality is trivially 1.0 because all agents receive near-zero reward.

**Maximin optimization rescues efficiency.** Strikingly, under maximin optimization the researcher achieves *higher* efficiency ( $U = 1.71$ ) than under direct efficiency optimization ( $U = 0.87$ ), alongside strong equality ( $E = 0.94$ ) and positive worst-off welfare ( $\min_i R_i = 137$ ). This reversal, absent in frontier models where both objectives yield  $U \approx 3.1$ – $3.2$ , occurs because the maximin objective forces the researcher toward coordination mechanisms (rotating cleaning duties, index-based apple assignment) that simultaneously improve collective output. Under efficiency-only optimization, the researcher converges on a local optimum that a 26B model can implement but that caps performance well below the frontier.

This suggests that *for models below a capability threshold, fairness objectives may serve as better optimization targets for overall social welfare than direct efficiency maximization*. The structured coordination enforced by the maximin objective provides a scaffold that compensates for the weaker model’s difficulty implementing complex strategies from strategic hints alone.

**Gathering: full recovery on a simpler game.** On Gathering ( $N=4$ ), the researcher brings Gemma from  $U=0.0$  to  $U=2.44$  with  $E=0.98$  and  $\min_i R_i=580$ , after 10 outer iterations (4 kept). These values nearly match frontier models (Gemini:  $U=2.49$ , Sonnet:  $U=2.52$ ; Table 2). The researcher discovers the same Voronoi partitioning and respawn-aware camping strategies found in the main experiments, and increases inner-loop iterations to  $K=5$  to allow sufficient refinement. The contrast with Cleanup suggests that the researcher can fully compensate for model weakness on simpler coordination tasks (4 agents, symmetric costs) but not on harder provision dilemmas (10 agents, asymmetric costs).

## A.2 Compute Requirements

Table 5 reports wall-clock time for all 12 autoresearch runs. *Inner loop* measures total time spent on policy LLM generation and environment simulation across all evaluations; *total* (estimated from run-directory timestamps) additionally includes the researcher agent’s analysis, code editing, and decision-making between evaluations.

Table 5: Wall-clock time per autonomous run. Evals: total inner loop evaluations including initial baseline. Per eval: mean wall-clock per single evaluation. Total: end-to-end including researcher overhead.

| Policy LLM                 | $\Phi$        | Evals | Inner loop (h) | Per eval (min) | Total (h) |
|----------------------------|---------------|-------|----------------|----------------|-----------|
| <i>Cleanup</i> ( $N=10$ )  |               |       |                |                |           |
| Gemini                     | $\Phi_U$      | 11    | 2.6            | 14             | 3.7       |
| Gemini                     | $\Phi_U$      | 18    | 4.1            | 14             | 6.4       |
| Sonnet                     | $\Phi_U$      | 4     | 2.1            | 32             | 6.1       |
| Sonnet                     | $\Phi_U$      | 7     | 5.0            | 43             | 6.1       |
| Gemini                     | $\Phi_{\min}$ | 17    | 3.0            | 11             | 4.3       |
| Gemini                     | $\Phi_{\min}$ | 11    | 3.6            | 20             | 5.0       |
| Sonnet                     | $\Phi_{\min}$ | 8     | 4.5            | 33             | 8.8       |
| Sonnet                     | $\Phi_{\min}$ | 9     | 5.3            | 36             | 13.2      |
| <i>Gathering</i> ( $N=4$ ) |               |       |                |                |           |
| Sonnet                     | $\Phi_U$      | 3     | 1.7            | 33             | 2.2       |
| Gemini                     | $\Phi_U$      | 5     | 1.4            | 17             | 1.9       |
| Gemini                     | $\Phi_U$      | 3     | 0.9            | 19             | 1.1       |
| Sonnet                     | $\Phi_U$      | 4     | 2.3            | 35             | 3.4       |
| All 12 runs                |               | 100   | 36.6           | 22             | 62.2      |

**Cost breakdown.** Each inner loop evaluation involves  $K=2-3$  policy LLM generation calls (each  $\sim 2k$  input tokens,  $\sim 1.5k$  output tokens) followed by multi-seed simulation (5–12 seeds,  $\sim 15-30s$  total). Policy LLM generation dominates inner loop time (86–97%), with Sonnet evaluations taking  $\sim 2\times$  longer than Gemini due to extended thinking. The researcher agent (Claude Opus 4.6, running via Claude Code CLI) accounts for 41% of total wall-clock time (25.6h of the 62.2h total across all 12 runs), spent reading results, editing pipeline source files, and planning modifications. In monetary terms, the researcher agent is the dominant cost: each outer iteration consumes  $\sim 50-100k$  context tokens in the Opus session, whereas each inner loop evaluation uses only  $\sim 5-10k$  policy LLM tokens.

## B Researcher-Authored Pipeline Artifacts

The previous appendix shows the policies  $\pi_c^*$  that the inner loop *outputs*; this one shows the configuration  $c = (p, \phi, \mathcal{H}, \iota)$  (Section 4, Table 1) that the researcher  $\mathcal{R}$  *authored* to produce them. Excerpts are taken from the final commit on the dedicated git branch of the corresponding run; we reproduce

the prose verbatim, with author commentary in [brackets] and elisions marked . . . . We organize by artifact type so each subsection makes a within-type contrast (e.g.,  $\Phi_U$  vs.  $\Phi_{\min}$ , or Cleanup vs. Gathering). Subsection B.1 comes first because it shows the upstream input that the rest of the appendix is derived from: the prompt  $p_{\mathcal{R}}$  given to the researcher itself.

## B.1 Researcher system prompt $p_{\mathcal{R}}$

The researcher  $\mathcal{R}$  is a coding agent (Claude Opus 4.6 via the Claude Code CLI) instantiated with a single fixed system prompt that is identical across all experiment runs (it differs only in the game name and the primary-metric flag, which is one of efficiency or maximin). This is the input from which every artifact in the rest of this appendix is derived. Because much of the paper’s claim hinges on the asymmetry between *what  $p_{\mathcal{R}}$  tells  $\mathcal{R}$*  and *what  $\mathcal{R}$  in turn writes for  $\mathcal{M}$* , we reproduce the strategically relevant portions of  $p_{\mathcal{R}}$  verbatim below; the full file is in the released repository under autoresearch/program.md.

Listing 1: Excerpt from  $p_{\mathcal{R}}$ . The remaining parts of  $p_{\mathcal{R}}$  describe the file layout of the inner-loop pipeline, the evaluation script invocation, the keep/discard rule, and the logging format; none of those add task-specific strategic guidance.

```

465  ## The metrics
466
467  The primary metric is specified at launch via '--metric' (default: 'efficiency').
468  The two options are:
469
470  Efficiency (U): collective apple collection rate across all agents per
471  timestep. Higher is better. ...
472
473  Maximin (Rawlsian welfare): minimum total per-agent return across all
474  agents. Higher is better. Inspired by Rawls' difference principle -- a just
475  policy maximizes the welfare of the worst-off agent. ...
476
477  ## Strategy space
478
479  Here are categories of modifications to explore:
480
481  ### Prompt engineering (p)
482  - Add strategic hints about the Cleanup dilemma (e.g., "cleaning is a public
483    good -- someone must do it")
484  - Add worked examples of sophisticated policies
485  - Restructure the API documentation for clarity
486  - Add game-theoretic reasoning frameworks
487  - Mention optimal strategies from the literature (Voronoi partitioning,
488    role assignment)
489
490  ### Feedback engineering (l, phi)
491  - Show per-agent reward breakdown (not just average)
492  - Add derived metrics (e.g., cleaning rate, waste level trends, apple growth)
493  - Frame feedback to emphasize cooperation
494  - Add temporal analysis ...
495  - Show metric trends across iterations ...
496  - Provide diagnostic hints based on metrics ...
497
498  ### Helper functions (H)
499  - count_waste(env), waste_fraction(env), bfs_to_waste(env, agent_id),
500    should_clean(env), assign_role(env, agent_id),
501    find_cleaning_position(env, agent_id)
502
503  ### Iteration logic (iota)
504  - Change K (more iterations = more refinement but more cost)
505  - Change eval seeds, retry budget, thinking budget
506

```

**What  $p_{\mathcal{R}}$  does *not* say.** For the convergent-discovery claim of Finding 4 to be informative,  $p_{\mathcal{R}}$  must not itself encode the specific mechanisms that  $\mathcal{R}$  later writes into  $p$ ,  $\phi$ ,  $\mathcal{H}$  under  $\Phi_{\min}$ . Inspecting Listing 1, three absences matter:

- *No rotation formula.*  $p_{\mathcal{R}}$  never mentions  $(\text{agent\_id} + \text{env}.\_ \text{step\_count} // T) \% n$ ,  $\text{env}.\_ \text{step\_count}$ , or any other time-based cycling pattern. The closest item – “Mention optimal strategies from the literature (Voronoi partitioning, role assignment)” – names *static* role assignment, not time-rotated duty.
- *No objective-conditional guidance.* The strategy-space listing is identical regardless of whether  $\mathcal{R}$  is launched with `-metric efficiency` or `-metric maximin`.  $p_{\mathcal{R}}$  does not tell  $\mathcal{R}$  to behave

517 differently under the two objectives; the only objective-sensitive input is the (scalar) score  $\Phi_j$   
 518 returned after each inner-loop run.  
 519 • No “rotation is for fairness.”  $p_{\mathcal{R}}$  does not link any strategy to either welfare criterion. The  
 520 connection “rotation = fairness = high maximin” is constructed by  $\mathcal{R}$  from  $\Phi_j$  observations across  
 521 iterations on the branch.

522 The researcher-authored prompt  $p$  shown in Listing 4 and its Sonnet counterpart, both written under  
 523  $\Phi_{\min}$ , contain all three of these elements; the corresponding  $\Phi_U$  prompts do not. The role of  $p_{\mathcal{R}}$  in  
 524 the experiment is therefore to define the configuration space and the evaluation harness, not to script  
 525 the discoveries themselves.

## 526 B.2 Helper library $\mathcal{H}$ : coordination primitives

527 The researcher adds primitives that the policy LLM can call as black boxes, sparing it from re-  
 528 implementing tricky logic on every iteration. Two patterns dominate: (i) *state inspection* (waste  
 529 counts, fractions, beam-yield scoring) and (ii) *coordination primitives* (zone assignment, role rota-  
 530 tion). We show one of each.

Listing 2: Cleanup,  $\Phi_{\min}$  – band-based apple zoning helper added by the researcher in exp5. This is  
 the primitive that lets the policy in Listing 7 keep collectors within their own row band, preventing  
 all 7 gatherers from racing to the same apple.

```
531 1 def get_my_apples(env, agent_id):
532 2     """Alive apples in this agent's horizontal row band.
533 3     Divides the orchard into n_agents bands; falls back to all apples if empty."""
534 4     aid = int(agent_id)
535 5     n = int(env.n_agents)
536 6     band_h = env.height / n
537 7     band_lo, band_hi = aid * band_h, (aid + 1) * band_h
538 8
539 9     my_apples, all_apples = set(), set()
540 10    for i in range(env.n_apples):
541 11        if env.apple_alive[i]:
542 12            ar = int(env._apple_pos[i, 0]); ac = int(env._apple_pos[i, 1])
543 13            all_apples.add((ar, ac))
544 14            if band_lo <= ar < band_hi:
545 15                my_apples.add((ar, ac))
546 16    return my_apples if my_apples else all_apples
```

Listing 3: Gathering – BFS-Voronoi territory + respawn-aware waiting helpers added by the re-  
 searcher in gather-exp3. These are the primitives that Listing 9 composes into a one-line policy: “go  
 to my\_zone\_apples, otherwise nearest\_respawning\_apple.”

```
549 1 def voronoi_zones(env):
550 2     """Multi-source BFS Voronoi over walkable cells.
551 3     Returns (row, col) -> agent_id; ties broken by lower agent_id."""
552 4     queue, visited = deque(), {}
553 5     for a_id in range(env.n_agents):
554 6         if int(env.agent_timeout[a_id]) == 0:
555 7             ar = int(env.agent_pos[a_id][0]); ac = int(env.agent_pos[a_id][1])
556 8             queue.append((ar, ac, a_id))
557 9             visited[(ar, ac)] = a_id
558 10    while queue:
559 11        r, c, a_id = queue.popleft()
560 12        for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
561 13            nr, nc = r + dr, c + dc
562 14            if 0 <= nr < env.height and 0 <= nc < env.width \
563 15                and not env.walls[nr, nc] and (nr, nc) not in visited:
564 16                visited[(nr, nc)] = a_id
565 17                queue.append((nr, nc, a_id))
566 18    return visited
567 19
568 20 def nearest_respawning_apple(env, agent_id, zones=None, max_timer=10):
569 21     """Nearest dead apple in MY zone respawning within max_timer steps.
570 22     Returns (row, col) of best wait spot, or None."""
571 23     if zones is None: zones = voronoi_zones(env)
572 24     best_pos, best_t, best_d = None, max_timer + 1, float('inf')
573 25     ar = int(env.agent_pos[agent_id][0]); ac = int(env.agent_pos[agent_id][1])
574 26     for i in range(env.n_apples):
575 27         if not env.apple_alive[i]:
576 28             pos = (int(env._apple_pos[i][0]), int(env._apple_pos[i][1]))
577 29             if zones.get(pos) == agent_id:
578 30                 t = int(env.apple_timer[i])
579 31                 if t <= max_timer:
```



```

581 32         d = abs(pos[0] - ar) + abs(pos[1] - ac)
582 33         if t < best_t or (t == best_t and d < best_d):
583 34             best_pos, best_t, best_d = pos, t, d
584 35     return best_pos

```

586 The choice of helper depends on the dilemma’s geometry: row-bands suffice when fairness is the  
587 main concern (Cleanup maximin) but full BFS-Voronoi is needed when wall-aware territory owner-  
588 ship matters (Gathering). The researcher does not fix this in advance, it picks the simpler primitive  
589 whenever it works.

### 590 B.3 System prompt $p$ : from neutral framing to strategic briefing

591 The unmodified system prompt is a neutral API description: “Write a policy that maximizes per-  
592 agent reward.” Under maximin optimization, the researcher rewrites the prompt into a multi-section  
593 strategic briefing. The excerpt below shows the pieces it added to the Cleanup prompt in exp5; the  
594 full prompt grew from 165 to 325 lines.

Listing 4: Cleanup,  $\Phi_{\min}$  – excerpts from the researcher-rewritten system prompt (exp5). The unmodified baseline contained only API documentation; everything below was added by  $\mathcal{R}$ .

```

595 ## CRITICAL OBJECTIVE: Rawlsian Fairness (Maximin)
596
597 Your goal is to maximize the minimum per-agent total return across all
598 agents. This is the "maximin" or Rawlsian welfare criterion.
599
600 This means: it is NOT enough for the *average* agent to do well. The
601 worst-off agent must do as well as possible. A policy where 8 agents
602 each earn +200 but 2 agents each earn -100 is TERRIBLE (maximin = -100).
603
604 Key implication: cleaning costs (-1 per CLEAN action) must be shared
605 equitably among ALL agents. If you assign fixed "cleaner" roles, those
606 agents will accumulate large negative rewards and destroy your maximin score.
607
608 ## Strategy for Maximin: ROLE ROTATION + APPLE ZONING
609
610 The optimal strategy for maximin involves:
611 1. Shared cleaning duty: ALL agents take turns cleaning using a rotation
612    schedule based on 'agent_id' and 'env._step_count'. For example:
613    'is_my_cleaning_turn = (agent_id + env._step_count // SHIFT) % env.n_agents < NUM_CLEANERS'
614    where SHIFT is -50 steps and NUM_CLEANERS is 2-3.
615 2. When NOT your turn: collect apples from YOUR ZONE using
616    'get_my_apples(env, agent_id)' -- prevents all gatherers competing
617    for the same nearest apple.
618 3. NEVER use BEAM (action 6): it costs -1 and causes -50 to the target.
619 4. Keep waste density low: aim for ~2-3 active cleaners at any time.
620
621 [... API documentation, helper documentation, working example ...]
622
623 IMPORTANT:
624 - NEVER assign permanent cleaning roles to specific agents -- this kills maximin.
625 - Use env._step_count for time-based role rotation so all agents share cleaning duty.
626 - NEVER use BEAM (action 6) -- it destroys both agents' rewards.
627

```

629 Two things stand out. First, the researcher writes the formula it expects  $\mathcal{M}$  to use almost verbatim  
630 ( (agent\_id + env.\_step\_count // SHIFT) % env.n\_agents < NUM\_CLEANERS ) and the  
631 policy in Listing 7 uses essentially this template. Second, the researcher *teaches by counter-example*:  
632 the explicit “8 agents earn +200, 2 earn -100, maximin = -100” makes the failure mode of  $\Phi_U$ -style  
633 static roles (Listing 6) concretely visible to  $\mathcal{M}$ . This is the information-design move predicted by  
634 the mechanism-design framing in Section 3.4.

635 The Gathering prompt evolves along a different axis (no maximin, no rotation). Its researcher-added  
636 “Key Strategic Insights” section instead emphasizes (i) *self-play implies never beam* and (ii) *Voronoi*  
637 *partition each step*, exactly matching what the policy in Listing 9 implements.

### 638 B.4 Feedback $\phi$ : adaptive diagnostics

639 The baseline feedback function from [3] shows reward + four social metrics with definitions and  
640 stops there. The researcher turns it into a state machine: it inspects the latest metrics and condition-  
641 ally injects different hints. Two different runs produced two qualitatively different diagnostics: the

642 same metric ( $\bar{r}_k$  or  $\min_i R_i$ ) is repurposed to either *stabilize* a working policy or *redirect* a failing  
643 one.

Listing 5: Adaptive feedback diagnostics. *Top*: efficiency-optimized run (exp1) – a stability guard prevents iter-3 regression once  $U$  is high. *Bottom*: maximin-optimized run (exp5) – two fairness diagnostics that fire when the worst-off agent loses reward.

```

644 1 # --- Cleanup, Phi_U feedback (exp1, pipeline/feedback.py final) ---
645 2 last_eff = history[-1]["metrics"]["efficiency"]
646 3 if last_eff >= 2.5:
647 4     parts.append(
648 5         """CRITICAL -- DO NOT REGRESS*: The current policy achieves high "
649 6         "efficiency (>=2.5). You MUST output a policy that is nearly identical "
650 7         "to the current one. Copy the current policy code and make AT MOST "
651 8         "one small targeted improvement (e.g., adjust a single numeric "
652 9         "threshold). If you are not confident a change will help, output the "
653 10        "current policy UNCHANGED. A regression here means the run fails.")
654 11
655 12 # --- Cleanup, Phi_min feedback (exp5, pipeline/feedback.py final) ---
656 13 last_maximin = history[-1]["metrics"].get("maximin", 0)
657 14 last_avg = history[-1]["reward_avg"]
658 15 if last_maximin < 0:
659 16     parts.append(
660 17         f"""FAIRNESS ALERT*: maximin={last_maximin:.1f} is NEGATIVE. "
661 18         f"The worst-off agent lost reward overall while average was {last_avg:.1f}. "
662 19         "This means cleaning duties are NOT shared equitably. "
663 20         "Use time-based role rotation (env._step_count) so ALL agents share "
664 21         "cleaning costs equally. NEVER assign permanent cleaner roles.")
665 22 elif last_maximin < last_avg * 0.5:
666 23     parts.append(
667 24         f"""FAIRNESS WARNING*: maximin={last_maximin:.1f} is much lower than "
668 25         f"average={last_avg:.1f}. The gap suggests unequal cleaning burden. "
669 26         "Ensure ALL agents rotate through cleaning duty.")
670 27

```

672 The two diagnostics are written by independent researcher runs but converge on a common pattern:  
673 *thresholded interventions on a primary metric*. They are also a common cause of the low run-to-  
674 run variance reported in Finding 1: when a working policy lands in the high- $U$  basin, the stability  
675 guard prevents the LLM from over-refining and tipping out of it (the  $K=3$  regressions visible in  
676 Section 4’s “Common failure modes” (4)), and when an unfair policy lands in the negative-maximin  
677 region, the alert injects exactly the rotation hint that recovers it.

## 678 B.5 Iteration logic $\iota$ : per-condition hyperparameters

679 Although  $\iota$  has the smallest source surface, the researcher’s edits to it explain a meaningful fraction  
680 of the variance reduction noted in Finding 1. Table 6 summarizes the values shipped in the final  
681 commit of each best-performing run.

Table 6: Iteration-logic ( $\iota$ ) values in the final commit of selected runs.  $K$  = inner-loop iterations,  $|S|$  = evaluation seeds, “thinking” = extended-thinking token budget passed to  $\mathcal{M}$ . Baseline ([3]) is  $K=3$ ,  $|S|=5$ , thinking 16k.

| Run                  | Game / objective       | $K$      | $ S $     | thinking   | Researcher’s stated rationale                                           |
|----------------------|------------------------|----------|-----------|------------|-------------------------------------------------------------------------|
| exp1 (Gemini)        | Cleanup, $\Phi_U$      | 3        | 5         | 16k        | default; stability comes from feedback hint                             |
| exp4 (Sonnet)        | Cleanup, $\Phi_U$      | 3        | 5         | 32k        | “Sonnet was over-refining at 16k thinking”                              |
| exp5 (Gemini)        | Cleanup, $\Phi_{\min}$ | <b>2</b> | <b>12</b> | 16k        | “ $K=2$ avoids iter-3 regression; 12 seeds for variance control”        |
| exp7 (Sonnet)        | Cleanup, $\Phi_{\min}$ | 3        | 5         | <b>10k</b> | “reduced thinking from 16k – Sonnet was generating runaway code at 16k” |
| gather-exp3 (Gemini) | Gathering, $\Phi_U$    | 3        | 5         | 16k        | default; Gathering converges in $K \leq 3$                              |

682 The maximin Gemini run is the most informative case: the researcher discovered that with  $K=3$   
683 and  $|S|=5$ , identical configurations could yield  $\min_i R_i=295$  on one set of seeds and  $\min_i R_i=100$   
684 on another (this is the “variance exposure” failure mode of Section 4’s common failures). It then  
685 walked  $|S|$  from  $5 \rightarrow 8 \rightarrow 12$  over three outer iterations until the seed-averaged signal was stable  
686 enough that the policy LLM stopped chasing noise. The policy in Listing 7 is sampled at  $|S|=12$ .

## 687 B.6 Cross-artifact observations

- 688 • *Helpers do the algebra; prompts pick the strategy class*. The researcher uses helpers (2, 3) to  
689 put non-trivial primitives at  $\mathcal{M}$ ’s fingertips, and the prompt (4) to tell  $\mathcal{M}$  *which* primitive to

690 compose. Prompts without supporting helpers produced policies that “mention rotation” but failed  
 691 to implement it; helpers without prompt updates often went unused.  
 692 • *Feedback is the only adaptive component.*  $p$ ,  $\mathcal{H}$ , and  $\iota$  are static within a run;  $\phi$  is the only place  
 693 where the researcher gets to change what  $\mathcal{M}$  sees *during* the inner loop, and it uses thresholded  
 694 diagnostics (Listing 5) to do so. This is what mostly drives the run-to-run variance reduction.  
 695 • *The same artifact has different content under each objective.* The Cleanup prompt under  $\Phi_U$  omits  
 696 “rotation” and “maximin” entirely; the same prompt under  $\Phi_{\min}$  devotes its entire opening section  
 697 to it. The configuration  $c$  is genuinely a function of the welfare objective  $\Phi$ , not a fixed pipeline  
 698 parameterized by it. This is consistent with the information-design framing of Section 3.4.  
 699 • *The researcher’s edits are short and targeted.* The full diff between the baseline pipeline and the  
 700 best maximin Gemini configuration totals  $\sim 300$  added lines spread over four files;  $\sim 80$  of them  
 701 appear in the listings above. The remaining content is API documentation and worked examples,  
 702 included for completeness but not strategically novel.

## 703 C Selected Generated Policies

704 The code excerpts below are verbatim outputs of the policy synthesizer  $\mathcal{M}$ , taken from the final  
 705 inner-loop iteration of the best-performing run in each condition. To fit the page, we elide repeated  
 706 boilerplate (BFS scaffolding, fallback branches) with  $\dots$  and add commentary in [brackets];  
 707 nothing else has been edited. The listings illustrate the qualitative differences highlighted in Findings  
 708 2–4 of Section 4: *static* vs. *rotating* cleaning roles in Cleanup, and the *Voronoi* + *respawn-timer*  
 709 structure in Gathering. All four policies were independently generated by the LLM under different  
 710 seeds, models, and outer-loop runs.

### 711 C.1 Cleanup, $\Phi_U$ (Gemini): static interleaved roles + dynamic threshold

712 Best run:  $U=3.25$ ,  $E=0.61$ ,  $\min_i R_i = -182$ . The synthesizer assigns *permanent* cleaning roles via  
 713  $\text{agent\_id} \bmod n < \tau$ , where the threshold  $\tau$  depends smoothly on the current waste fraction. Higher  
 714 pollution recruits more cleaners, but the *identity* of each cleaner is fixed: agents  $0\text{--}\tau-1$  always pay  
 715 the cost. This drives  $U$  to its ceiling but leaves equality at 0.61 and pushes the worst-off agent to  
 716  $\min_i R_i = -182$ .

Listing 6: Cleanup,  $\Phi_U$  – Gemini 3.1 Pro. Static role assignment with waste-tiered threshold (ver-  
 batim, abridged).

```

717 1 def policy(env, agent_id) -> int:
718 2     if int(env.agent_timeout[agent_id]) > 0:
719 3         return 7 # STAND while removed
720 4
721 5     waste_frac = waste_fraction(env)
722 6     n_agents = env.n_agents
723 7
724 8     # [Smooth dynamic threshold: more pollution -> more cleaners]
725 9     if waste_frac > 0.35: threshold_cleaners = n_agents
726 10    elif waste_frac > 0.25: threshold_cleaners = int(n_agents * 0.7)
727 11    elif waste_frac > 0.15: threshold_cleaners = int(n_agents * 0.5)
728 12    elif waste_frac > 0.05: threshold_cleaners = int(n_agents * 0.3)
729 13    else: threshold_cleaners = max(1, int(n_agents * 0.2))
730 14
731 15    # [STATIC role: cleaners are ALWAYS the lowest-id agents -- no rotation]
732 16    is_cleaner = (agent_id % n_agents) < threshold_cleaners
733 17
734 18    if is_cleaner:
735 19        best_orient, n_waste = find_best_clean_orientation(env, agent_id)
736 20        if n_waste >= 1:
737 21            cur_orient = int(env.agent_orient[agent_id])
738 22            if best_orient == cur_orient: return 8 # CLEAN
739 23            if (cur_orient + 1) % 4 == best_orient: return 5 # ROTATE_RIGHT
740 24            elif (cur_orient - 1) % 4 == best_orient: return 4 # ROTATE_LEFT
741 25            else: return 4
742 26            # [Competitor-aware BFS toward waste in own row band]
743 27            c_idx = agent_id % n_agents
744 28            target_r = int(((c_idx + 0.5) / threshold_cleaners) * env.height)
745 29            # ... pick waste cell minimizing dist + |row - target_r| + 50 *
746 30            # closer_cleaners
747 31            # ... bfs_toward and direction_to_action
748 32        else:
749 33            # GATHERER: row-banded sweep, competitor-aware
750 34            # assigned_row = ((g_idx + 0.5) / n_gatherers) * env.height
751 35

```

```

752 34 # ... pick alive apple minimizing dist + /row - assigned_row/ + 50 *
753      #closer_gatherers
754 35 ...

```

## 756 C.2 Cleanup, $\Phi_{\min}$ (Gemini): time-rotated, geographically interleaved roles

757 Best run:  $U=3.19$ ,  $E=0.98$ ,  $\min_i R_i=296$ . Switching the outer objective from  $U$  to  $\min_i R_i$  flips  
758 the role-assignment scheme entirely: `role_idx` now depends on `agent_id + step/50`, so every  
759 agent rotates through cleaner and gatherer duty in a 50-step cycle. The cleaner indices  $\{1, 4, 8\}$  are  
760 spatially *interleaved* along the river (rather than contiguous) so that whoever is on duty has a short  
761 walk to their assigned river slice. This single change accounts for the  $E=0.55 \rightarrow 0.98$  jump and the  
762  $\min_i R_i=-182 \rightarrow 296$  swing reported in Table 2, with negligible efficiency loss.

Listing 7: Cleanup,  $\Phi_{\min}$  – Gemini 3.1 Pro. Time-rotated roles with interleaved geography (verbatim, abridged).

```

763 1 def policy(env, agent_id) -> int:
764 2     if int(env.agent_timeout[agent_id]) > 0:
765 3         return 7 # STAND while removed
766 4
767 5
768 6     aid = int(agent_id)
769 7     step = int(env._step_count)
770 8     n = int(env.n_agents)
771 9
772 10    # [TIME ROTATION: every 50 steps each agent's role index advances]
773 11    if n == 10:
774 12        role_idx = (aid + step // 50) % 10
775 13        # [Cleaner indices {1,4,8} are SPATIALLY INTERLEAVED, not contiguous,
776 14        # so each on-duty cleaner has a short walk to its river slice]
777 15        if role_idx == 1: role_type, zone_idx = 'C', 0
778 16        elif role_idx == 4: role_type, zone_idx = 'C', 1
779 17        elif role_idx == 8: role_type, zone_idx = 'C', 2
780 18        else:
781 19            role_type = 'G'
782 20            g_map = {0:0, 2:1, 3:2, 5:3, 6:4, 7:5, 9:6}
783 21            zone_idx = g_map[role_idx]
784 22            num_cleaners, num_gatherers = 3, 7
785 23
786 24    if role_type == 'C':
787 25        # [Each on-duty cleaner owns one horizontal slice of the river]
788 26        row_min = int( zone_idx * env.height / num_cleaners)
789 27        row_max = int((zone_idx + 1) * env.height / num_cleaners) - 1
790 28
791 29        # [Score every (row, col, orientation) in the slice by beam yield;
792 30        # fire if facing >=2 waste, else move to a >=2-yield position]
793 31        best_positions, good_positions = set(), set()
794 32        for r in range(row_min, row_max + 1):
795 33            for c in range(10):
796 34                if env.walls[r, c] or env.waste[r, c]: continue
797 35                y_max = max(get_clean_yield(env, r, c, o) for o in range(4))
798 36                if y_max >= 2: best_positions.add((r, c))
799 37                if y_max >= 1: good_positions.add((r, c))
800 38            # ... rotate / CLEAN / bfs_to_target_set as appropriate
801 39        else:
802 40            # GATHERER: collect apples ONLY inside the rotating row band
803 41            # row_min/row_max for num_gatherers slices ...
804 42            ...

```

## 806 C.3 Cleanup, $\Phi_{\min}$ (Sonnet): independent rediscovery of duty rotation

807 This run was launched on a separate dedicated git branch from the Gemini maximin runs of List-  
808 ing 7 and from  $p_{\mathcal{R}}$  identical to the one shown in Appendix B.1, with no shared state. The researcher-  
809 authored synthesizer prompt  $p$  on this branch ended up containing a rotation hint structurally similar  
810 to Listing 4 but with different phrasing and a different recommended period (`phase_length = 100`  
811 in a worked example, vs. `SHIFT  $\approx$  50` in Listing 4);  $\mathcal{M}$  then chose its own period of 50. The con-  
812 vergence is therefore at the level of which artifacts  $\mathcal{R}$  injects into  $p$ , not at the level of  $\mathcal{M}$  inventing  
813 rotation from a neutral prompt.

814 Best run:  $U=2.93$ ,  $E=0.83$ ,  $\min_i R_i=154$ . The Sonnet synthesizer arrives at the *same* structural  
815 insight as the Gemini maximin run: a phase counter (`agent_id + step/50`) mod  $n$  rotates which  
816 2 of the 10 agents clean at any given time, and a *separate* 200-step zone counter rotates which 5-

row band each collector sweeps. The two rotation periods (50 and 200 steps) ensure every agent visits every cleaning slot and every apple zone within one episode, resulting in a structural fairness invariant.

**An alternative Sonnet maximin run uses a different mechanism.** The second Sonnet  $\Phi_{\min}$  run took a structurally distinct route. There,  $\mathcal{R}$  never wrote a rotation template into  $p$ ; instead it authored a “collective threshold” worked example in which *all* agents synchronously enter a cleaning mode whenever `waste_fraction(env) > 0.22` and a collecting mode when it drops below 0.08, with no `agent_id` phase shift. The resulting synthesizer policy follows this pattern and reaches  $\min_i R_i=200$ , slightly above the rotation policy of Listing 8, confirming that the convergence in Table 3 is on the broader *class* (explicit fairness mechanism, 4/4 maximin) rather than on rotation specifically.

Listing 8: Cleanup,  $\Phi_{\min}$  – Sonnet 4.6. Phase-rotated cleaning + zone-rotated collection (verbatim, abridged).

```

828 1 def policy(env, agent_id) -> int:
829 2     if int(env.agent_timeout[agent_id]) > 0:
830 3         return 7 # STAND while removed
831 4
832 5
833 6     n = env.n_agents
834 7     step = env._step_count
835 8     wf = waste_fraction(env)
836 9
837 10    # --- ROLE ASSIGNMENT: fair 2-cleaner rotation ---
838 11    # [50-step phases x 20 phases / episode -> each agent cleans 4 phases (~20%)]
839 12    phase = step // 50
840 13    cleaner_rank = (agent_id + phase) % n # [cycles 0..9 fairly]
841 14    is_cleaner = cleaner_rank < 2 # [exactly 2 cleaners per phase]
842 15
843 16    # [Emergency override: all agents clean when waste crosses the spawn cliff]
844 17    in_emergency = wf >= 0.40
845 18    if in_emergency: is_cleaner = True
846 19
847 20    # [River split: cleaner_slot 0 -> top half, 1 -> bottom half]
848 21    cleaner_slot = (agent_id % 2) if in_emergency else cleaner_rank
849 22
850 23    if is_cleaner and wf > 0.04:
851 24        # [Scan 4 orientations from current position; if best beam shot
852 25        # covers >=3 waste cells, fire; else step to a strictly better neighbor]
853 26        # ... (best_o, best_cnt) = max over 4 directions
854 27        # ... if best_cnt >= 3: rotate to best_o, then CLEAN
855 28        # ... else: scan 4 adjacent cells x 4 orientations for higher yield
856 29        # ... if no waste in beam range: bfs to waste_set in DOWN river half,
857 30        # fall back to whole river
858 31
859 32    # --- COLLECTOR LOGIC: 5-zone rotating apple collection ---
860 33    # [5 zones x 200 steps = 1000 steps -> each agent visits every zone once;
861 34    # 2 agents share each zone at any time, reducing competition 10 -> 2]
862 35    bounds = zone_boundaries(env, 5)
863 36    zone_phase = step // 200
864 37    zone = (agent_id + zone_phase) % 5
865 38    z_start, z_end = bounds[zone], bounds[zone + 1]
866 39
867 40    zone_apples = {(int(env._apple_pos[i][0]), int(env._apple_pos[i][1]))
868 41                  for i in range(env.n_apples) if env.apple_alive[i]
869 42                  and z_start <= int(env._apple_pos[i][0]) < z_end}
870 43    # ... bfs_to_target_set(zone_apples), else bfs_nearest_apple as fallback
871 44    return 7
872 45

```

#### 873 C.4 Gathering (Gemini): wall-aware Voronoi + spatiotemporal targeting

Best run:  $U=2.47$ ,  $E=0.98$ ,  $\min_i R_i=580$ . In Gathering, where costs are symmetric, no role differentiation is needed: the entire optimization is spatial and temporal. The synthesizer (i) partitions cells into BFS-Voronoi territories owned by individual agents, (ii) runs a single centralized BFS from its own position to score every reachable cell with exact wall-aware distances, and (iii) for both alive and respawning apples in its territory, computes the *earliest collection time*  $\max(\text{walk\_dist}, \text{respawn\_timer})$  and targets the minimum. When the agent must wait on a dead apple, it steps onto a *non-spawn* adjacent cell rather than blocking a respawn point. This single sort key ( $\max(\text{distance}, \text{timer})$ ) subsumes both “go to nearest apple” and “camp respawn” as special cases.

Listing 9: Gathering – Gemini 3.1 Pro. Voronoi territories + spatiotemporal priority targeting (verbatim, abridged).

```

883 1 def policy(env, agent_id) -> int:
884 2     if int(env.agent_timeout[agent_id]) > 0:
885 3         return 7
886 4
887 5
888 6     r, c = int(env.agent_pos[agent_id][0]), int(env.agent_pos[agent_id][1])
889 7     orient = int(env.agent_orient[agent_id])
890 8
891 9     # [Wall-aware Voronoi over the gridworld; spawn_points = all apple cells]
892 10    zones = voronoi_zones(env)
893 11    spawn_points = {(int(p[0]), int(p[1])) for p in env._apple_pos}
894 12
895 13    # [Single centralized BFS: exact distance + first move to every reachable cell]
896 14    distances, first_moves = {}, {}
897 15    q = deque([(r, c, 0, None)])
898 16    visited = {(r, c)}
899 17    while q:
900 18        cr, cc, d, fm = q.popleft()
901 19        distances[(cr, cc)] = d
902 20        first_moves[(cr, cc)] = fm
903 21        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
904 22            nr, nc = cr + dr, cc + dc
905 23            if 0 <= nr < env.height and 0 <= nc < env.width and not env.walls[nr,
906 24                nc]:
907 25                if (nr, nc) not in visited:
908 26                    visited.add((nr, nc))
909 27                    q.append((nr, nc, d + 1, fm if fm is not None else (dr, dc)))
910 28
911 29    # [Both alive AND respawning apples in MY zone, with their respawn timer]
912 30    my_zone_spawns = []
913 31    for i in range(env.n_apples):
914 32        pr, pc = int(env._apple_pos[i][0]), int(env._apple_pos[i][1])
915 33        if zones.get((pr, pc)) == agent_id and (pr, pc) in distances:
916 34            my_zone_spawns.append((pr, pc, int(env.apple_timer[i]), distances[(pr,
917 35                pc)]))
918 36
919 37    # [SPATIOTEMPORAL PRIORITY: earliest collection time first.
920 38    # score = max(walk_distance, respawn_timer); ties broken by sooner timer]
921 39    my_zone_spawns.sort(key=lambda x: (max(x[3], x[2]), x[2], x[3]))
922 40
923 41    for pr, pc, timer, dist in my_zone_spawns:
924 42        if timer == 0:
925 43            if dist == 0: return 7
926 44            fm = first_moves[(pr, pc)]
927 45            return direction_to_action(fm[0], fm[1], orient)
928 46        else:
929 47            # [Dead apple: camp on safe adjacent cell]
930 48            # ... pick nearest neighbor (nr,nc) that is NOT in spawn_points,
931 49            # navigate there and STAND while waiting for respawn
932 50            ...
933 51    # [Global poaching fallback if zone is empty: nearest alive apple anywhere]
934 52    ...

```

## 935 C.5 Cross-condition observations

936 Several patterns are visible across these four listings (and confirmed by inspecting the remaining 8  
937 runs not shown):

- 938 • *Role assignment encodes the welfare objective.* Static  $\text{agent\_id} < \tau$  (Listing 6) maximizes  
939  $U$  but harms  $\min_i R_i$ . Time-rotated  $(\text{agent\_id} + \text{step} // T) \% n$  (Listings 7, 8) maximizes  
940  $\min_i R_i$  at  $\leq 1\%$  efficiency cost (Gemini).
- 941 • *Convergent rediscovery across LLMs.* Gemini and Sonnet, on independent runs with separate  
942 dedicated git branches, both arrive at the  $(\text{id} + \text{phase}) \% n$  duty-rotation idiom with similar  
943 phase lengths ( $T \approx 50$  steps) in 3/4 maximin runs, and both omit any explicit fairness mechanism  
944 under efficiency. The remaining 1/4 maximin run (Sonnet) converges on a structurally distinct  
945 synchronized clean/collect mechanism with comparable maximin (Appendix C.3). The point of  
946 convergence is the *class* of mechanism ( $\mathcal{R}$  writes an explicit fairness mechanism into  $p$  under  
947  $\Phi_{\min}$ , never under  $\Phi_U$ ), not necessarily the rotation idiom itself.
- 948 • *Game structure dictates strategy class.* Cleanup policies are dominated by role-assignment logic  
949 (cleaner vs. gatherer); Gathering policies are dominated by territory partitioning and respawn-  
950 timer reasoning, with no role differentiation at all. The researcher does not need to be told which  
951 class of strategy to write.

952 • *Earliest-collection-time as a unifying score.* The Gathering policy's  $\max(\text{walk}, \text{timer})$  key (List-  
953 ing 9) is structurally identical across the 4 Gathering runs (both Gemini and Sonnet), differing only  
954 in how the Voronoi diagram is computed, e.g. by Manhattan approximation, fully BFS-based, or  
955 cached helper.